CI&T

Q2 2026

AI-DLC

AI-Driven Development Life Cycle

From vibe-coding to engineering discipline.

Open source · AWS Labs · github.com/awslabs/aidlc-workflows

Alexandre Canova · canova@ciandt.com

Agenda

20 min · 10 min Q&A

01
Context

- **The problem with AI today**
- What is vibe-coding?

02
What is AI-DLC?

- **Methodology, phases & comparison**
- vs. Vibe-Coding and Spec-Driven Dev

03
How it works

- **Three phases: Inception, Construction, Operations**
- Mob Elaboration & cross-functional collaboration

04
Why AI-DLC

- **Productivity, BCPs, quality, documentation**
- Getting started

Where does developer time actually go?

AI writes code. But writing code is not the whole job.

Requirements clarification	Teams spend hours aligning on WHAT before a line is written
Rework from ambiguity	Misunderstood scope means implementation is thrown away
Disconnected documentation	Docs written after the fact, already stale on arrival
Story creation & estimation	Manual effort every sprint, inconsistent BCP granularity
Debugging AI output blindly	No shared context means the agent guesses, the dev debugs

Result: fast output on day 1 — high rework, low traceability, no link to backlog.

Vibe-Coding

What most teams are doing today — and why it stalls.

Prompt — **Generate** — **Tweak** — **Repeat**

Works for prototypes

Solo exploration and throwaway proofs-of-concept. Useful for initial discovery.

Fails at team scale

No shared context, diverging interpretations, no audit trail between sessions.

Expensive over time

Velocity drops as scope grows. Rework from ambiguity eats all the early gains.

"Fast on day 1. Expensive by sprint 3."

What is AI-DLC?

A methodology that steers AI coding agents across the full SDLC — from business intent to production — with the human always in control.

AWS Labs

Open source (MIT-0)
AWS Labs

Tool-agnostic

Claude Code, Amazon Q, Cursor, Cline, Copilot

Not a prompt

A structured workflow methodology

Human in control

AI proposes. Human approves. Always.

AI-DLC exists because vibe-coding doesn't scale. It brings methodology to agentic development.

The Big Picture

Every stage follows the same pattern — AI proposes, human approves, then repeat.

AI creates a plan › — **AI seeks clarification ›** — **Human approves ›** — **AI executes**

↺ repeats at every stage, within every phase

WORKING PHASES

01 INCEPTION
WHAT & WHY

- Workspace detection — greenfield or brownfield?
- Requirements analysis — structured clarifying questions
- User story & unit creation — decomposed work items
- Application design — architecture and component plan

Mob Elaboration — team validates AI proposals together

02 CONSTRUCTION
HOW

- Functional design — detailed component specification
- Code generation — AI writes against the approved design
- Build & test — AI validates against defined criteria
- Human review — team inspects artifacts before advancing

Mob Construction — team reviews AI proposals in real time

Vibe-Coding vs. Spec-Driven Dev vs. AI-DLC

Spec-Driven Development (SDD) brings specs before code. AI-DLC goes further.

	Vibe-Coding	Spec-Driven Dev	AI-DLC
Approach	Prompt → iterate	Spec → plan → implement	Structured phases with approval gates
Team model	Individual / solo	Developer-centric, small teams	Cross-functional mob (PO, Dev, QA, Arch)
Collaboration	None built-in	None built-in	Mob Elaboration & Mob Construction
Documentation	None	Specs as files	Full artifacts: reqs, design, tests
Metrics	None	None	User stories + byproducts enable velocity metrics
Quality gates	Developer discipline	Checklist-based	Blocking constraints per phase
Ops coverage	None	None	Phase defined — deployment roadmap

SDD closes the spec gap. AI-DLC closes the team, metrics, quality, and lifecycle gap.

Why Specs Matter

The difference between hoping for the best and knowing what 'done' looks like.

Vibe Coding

the bad way

Human: "Build me a login page"

AI builds... something

Human: "No, not like that"

AI rebuilds... differently

Human: "Closer, but the styling is wrong"

Repeat forever

Problem: No definition of "done." The AI guesses. You react. Progress is random.

Spec-Driven Dev

better, but a hand-off

- Someone writes a detailed spec or PRD
- Spec is handed to an AI or developer to implement
- Implementer fills gaps with assumptions
- Reviewer finds mismatches between intent and implementation
- Back-and-forth to close the gap between intent and result

Problem: The spec writer and the implementer have different mental models. Hand-off creates gaps.

AI-DLC Elaboration

no hand-off — co-created

Spec co-created by Human + AI

Conversation, not a doc tossed over a wall — no interpretation gap

Machine-verifiable success criteria

Tests, types, thresholds — the AI checks itself

Same system defines and builds

Shared understanding, zero hand-off

Quality gates enforce automatically

Work continues until every criterion is met

Result: No interpretation gap. Spec is shared understanding. Clear criteria = autonomous AI.

Autonomy = f(Criteria Clarity)

The AI-DLC insight: Autonomy is a function of criteria clarity.

The team's responsibility isn't just to ask for features — it's to define what done means.

✕ VAGUE CRITERIA	✓ CLEAR CRITERIA
"The login page should be fast" What does 'fast' mean? The AI keeps asking. Can't work autonomously.	**"Login API responds in <200ms at p95 under 100 concurrent users"** Measurable. AI runs a load test and verifies — no questions needed.
"The UI should look nice" AI has no idea what 'nice' means to your team. Interpretation varies endlessly.	**"All pages pass WCAG AA contrast, use design system tokens, render at 375/768/1280px"** Verifiable. AI checks contrast ratios, validates tokens, tests breakpoints.

Clear criteria → **faster planning · fewer questions · predictable results · lower token cost**

Mob Elaboration & Mob Construction

AI-DLC's model for cross-functional collaboration — because coding is a consequence, not the starting point.

WHO IS IN THE ROOM

Product Owner

Defines intent, acceptance criteria, business priority

Tech Lead / Architect

Validates feasibility, proposes design constraints

Developer

Reviews units of work, flags technical complexity

QA Engineer

Shapes test strategy during requirements, not after

WHAT HAPPENS IN EACH PHASE

Mob Elaboration — Inception

The team fills AI-DLC's structured question files together. AI proposes requirements, stories, and design units. The mob validates, challenges, and approves before anything is built.

Mob Construction — Construction

The team reviews AI's architecture and code proposals in real time. Technical decisions are made collaboratively. No one developer holds the context alone.

Coding becomes execution

When specifications are explicit and AI-validated, code generation is a consequence — not an act of interpretation. Less guesswork. Less rework. Higher quality.

Full SDLC Coverage + Extensibility

Not a code generator — a development system.

Business Intent	Requirements	Architecture	Code & Tests	Deployment	Operations

EXTENSIONS — LAYER YOUR ORG'S RULES ON TOP

Security baseline

Included out of the box — enforced as a blocking constraint

Compliance

HIPAA, PCI-DSS, SOC 2 — template-ready extensions

Internal policies

Architecture standards, naming, coding practices — in plain markdown

Custom team rules

Any quality or governance rule your org defines

Blocking constraints mean stages cannot proceed until criteria are met — quality is enforced, not suggested.

Custom AI-DLC Extensions

Layer your organization's rules into the workflow — loaded only when the team opts in.

HOW IT WORKS

1 Scan

AI-DLC scans for *.opt-in.md files at Inception start

2 Ask

Mandatory opt-in questions presented to the team

3 Load

If "YES" — full rule set loaded into agent context

4 Enforce

Blocking constraints active in all downstream phases

ANATOMY OF AN EXTENSION

`*.opt-in.md` OPTIONAL

Defines the opt-in question shown during Requirements Analysis. Its presence is what makes an extension team-opt-in rather than always-on. When the team answers YES, the companion rule file is loaded into the agent context.

`*.md` *(rule file)* MANDATORY

Plain Markdown that gets injected into the agent context when opted in. Defines blocking constraints the AI must satisfy before any phase can advance — e.g. SECURITY-01 (encryption at rest/transit), PBT-02 (round-trip tests for serialization).

Getting Started

1 Download	2 Drop into your project	3 Start with a phrase
Get the latest release zip from github.com/awslabs/aidlc-workflows	Copy rules to your agent's folder (~2 shell commands per platform)	"Using AI-DLC, …" — the workflow activates automatically from there

Works with:

Claude Code	Codex	Cursor	GitHub Copilot	Cline	Kiro

Core tenets: No duplication · Methodology first · Agnostic to tool or model · Human always in the loop

CI&T

Key Takeaways

- AI-DLC is a structured methodology, not a prompt trick
- Covers WHAT, WHY, and HOW — the complete SDLC from intent to operations
- Mob Elaboration: the right people define specs together, before coding starts
- Generates auditable artifacts: stories with BCPs, design docs, tests, documentation
- Quality and compliance enforced as blocking constraints per phase
- The agent proposes. You approve. Control stays with the team.

Start with one project this sprint — measure the difference.

github.com/awslabs/aidlc-workflows